Langage C avancé : Séance 10

Sujets divers

Marc TAJCHMAN
@-mail : marc.tajchman@cea.fr

CEA - DES/ISAS/DM2S/STMF/LDEI

Généricité

Motivation

Pour calculer la racine carrée (approchée) d'un nombre, C propose plusieurs fonctions :

```
float sqrtf(float x);
double sqrt(double x);
long double sqrtl(long double x);
```

L'utilisateur doit choisir la fonction correcte suivant le type de valeur dont il veut calculer la racine carrée:

```
double a, ra;
ra = sqrt(a);

float b, rb;
rb = sqrtf(b);

long double c, rc;
rc = sqrtl(c);
```

Si l'utilisateur choisit une version non adaptée

Si par exemple on écrit :

```
float r, x = 0.3f;
r = sqrt(x);
```

Ce que la machine va réellement exécuter:

```
float r, x = 0.3f;

double temp1 = (double) x;
double temp2 = sqrt(temp1);
r = (float) temp2;
```

Donc:

- la machine crée 2 variables supplémentaires, invisibles pour l'utilisateur;
- plusieurs conversions de type (double <-> float) sont effectuées;
- ces conversions de type risquent d'introduire des erreurs d'arrondi.

Exemple: examiner et exécuter le code dans Exemples/0_initial

Mot clef _Generic

Pour éviter des erreurs du type vues à l'exemple précédent, C propose (depuis 2011), le mot clef _Generic qui permet au compilateur de choisir le traitement adapté au type d'argument

Dans l'exemple ci-dessous la macro TYPE_NAME renvoie le type de la valeur passée en argument :

```
#define TYPE_NAME(X) _Generic((X), \
    int: "entier", \
    char: "lettre", \
    double: "double", \
    default: "inconnu")

int main() {
  printf("Type name: %s\n", TYPE_NAME(42.42));
  printf("Type name: %s\n", TYPE_NAME('A'));
  return 0;
}
```

Voir dans Exemples/1_Generic, 2 exemples de généricité.

Mot clef _Generic (2)

Dans Exemples/2_Generic, on trouvera un exemple de macro SQRT qui appelle soit une fonction système, soit une fonction utilisateur suivant le type de l'argument fourni à SQRT.

Le mot-clef _Generic n'est pas très utilisé parce qu'il a plusieurs inconvénients:

- S'il y a une erreur dans cette partie du code, c'est souvent difficile à debugger/corriger
- _Generic est toujours utilisée dans une macro, avec tous les défauts des macros C
- _Generic propose une version limitée de la généricité (deux instances de fonctions qui suivent un modèle générique doivent avoir le même nombre de paramètres par exemple), voir la notion de template en C++

Rappels et compléments sur les macros

Rappels sur les macros

Les macros indiquent pour le préprocesseur qu'il faut remplacer un mot dans un fichier par un fragment de code.

Exemples:

```
#define PI 3.1415

double rayon = 2.5;
double aire = PI*r*r;
double perimetre = 2*PI*r;
```

sera remplacé par:

```
double rayon = 2.5;
double aire = 3.1415*r*r;
double perimetre = 2*3.1415*r;
```

Le préprocesseur est un outil indépendant du compilateur et contrôle pas la syntaxe C.

L'exemple Exemples/3_macros/main1.c illustre un cas "pathologique".

Rappels sur les macros (2)

Une macro agit dans un fichier à partir de la ligne où elle est définie.

Par exemple:

```
a = X;
#define X 3
b = X;
```

```
a = X;
b = 3;
```

Rappels sur les macros (3)

Une macro peut servir à définir une "fonction":

```
#define f(x) \
  x*x + 1

double a = 2.3;
double b = f(a);
```

```
double a = 2.3;
double b = a*a + 1;
```

Ne pas mettre d'espace après f dans #define f() …

```
#define f (x) \
  x*x + 1

double a = 2.3;
double b = f(a);
```

```
double a = 2.3;
double b = (x) x*x+1(a);
```

Provoquera un message d'erreur du compilateur

Rappels sur les macros (4)

Il vaut mieux entourer les arguments d'une fonction macro avec des parenthèses et même toute l'expression avec des parenthèses (pour être certain que l'ordre de opérations soit correct dans le code après remplacement des macros)

```
#define f(x) \
  x*x + 1

double a = 2.3
double b = f(a+1);
double c = f(a) * f(a+1);
```

```
double a = 2.3;
double b = a+1*a+1 + 1;
double c = a*a + 1 * a+1*a+1 + 1;
```

```
#define f(x) \
  ((x)*(x) + 1)

double a = 2.3
double b = f(a+1);
double c = f(a) * f(a+1);
```

```
double a = 2.3;
double b = ((a+1)*(a+1) + 1);
double c = ((a)*(a) + 1)
         + ((a+1)*(a+1) + 1);
```

Rappels sur les macros (5)

Il faut être prudent quand on utilise des macro fonctions avec des arguments qui sont utilisés plusieurs fois

"Vraie fonction":

```
1: int f(int x) {
2:   return x*x + 1;
3: }
4: int i = 2;
5: int n = f(++i);
```

Fonction macro:

```
1: #define f(x) \
2:  ((x)*(x) + 1)
3:
4: int i = 2;
5: int n = f(++i);
```

Que valent i et n dans les 2 cas après la ligne 5 ?

Essayez de prévoir leurs valeurs ou faites tourner le code dans le répertoire Exemples/3_macros

Rappels sur les macros (6)

Enfin le préprocesseur ne contrôle pas les types des arguments et de la valeur de retour d'une fonction macro.

(mais ce sera contrôlé par le compilateur lui-même)

Un exemple dans le répertoire Exemples/4_macros montre l'utilisation des macros pour générer des chaines de caractères et des symboles (ici des noms de fonction

Nombre variable d'arguments

- pour des fonctions C
- pour des macros

Nombre variable d'arguments dans une fonction C

Un exemple est la fonction printf prend un nombre d'arguments qui peut être différent entre 2 utilisations

```
printf("Le temps de calcul de %s est %f secondes\n", "partie1", 14.5);
printf("Le temps total de calcul est %f secondes\n", 20.5);
```

Une fonction avec un nombre variable d'argument contient toujours un premier argument qui sert à indiquer le début de la liste des arguments variables.

Une fonction qui reçoit un nombre d'arguments variable, ne connait pas ce nombre, il faut donc que celui qui appelle la fonction lui indique cette information.

Par exemple, printf prend toujours comme premier argument une chaine de caractères que printf parcourt pour connaitre le nombre (et le type) d'arguments à la suite

Définition d'une fonction avec un nombre variable d'arguments

L'écriture d'une fonction de ce type est similaire à ce qui suit:

```
#include <stdarg.h>

void calcul(T p, ...) {
   va_list args;
   va_start(args, p);

   /* parcours des arguments
      avec va_arg(args, type)); */

   printf("\n");
   va_end(args);
}
```

- La variable `args` (de type `va_list`) est similaire à un pointeur qui va parcourir les différents arguments de la fonction
- `va_start` positionne `args` sur le premier argument qui suit `p`
- `va_arg` donne la valeur de l'argument courant de type "`type`" et positionne `args` sur l'argument suivant
- Il est très important de donner le type correct de chaque argument de la liste variable
- `va_end` doit être appelée à la fin du parcours des arguments (libère la mémoire utilisée par `stdarg`)

Exemples

Le répertoire Exemples/5_stdarg contient 2 exemples d’une fonction qui calcule la moyenne d’un ensemble de valeurs:

- La première fonction utilise le premier argument pour connaitre le nombre de valeurs dont il faut calculer la moyenne, les arguments suivants sont les valeurs
- La seconde fonction a l'ensemble des valeurs en arguments, suivi d'un argument avec une valeur "spéciale" ou "impossible" qui sert à indiquer la fin de la liste de valeurs. Ce dernier argument est parfois appelé "sentinelle".

C'est un procédé similaire à l'ajout du caractère "\0" qui sert à marquer la fin d'une chaine de caractères C.

Macro C avec un nombre variable d'arguments

L'utilisation principale consiste à appeler une "vraie fonction" en lui transmettant les paramètres variables (une macro locale __VA_ARGS__ qui contient les paramètres variables, est definie dans la macro eprintf) et en complétant les arguments (ici le nom du fichier source __FILE__ et le numéro de ligne __LINE___).

Une définition possible de type de macro:

```
#define eprintf (format, ...)  {        \
   fprintf(stderr, "%s:%d: "format,     \
      __FILE__, __LINE__, __VA_ARGS__) \
   exit(-1);                            \
}
```

Un exemple se trouve dans le fichier Exemples/5_stdargs/main3.c